

NewsIQ — AI Fake News Detector

Web Application Project Report — REST API / AI Integration Project

TASK-3

Prepared by: Manogna Yara

1. Introduction

NewsIQ is an AI-powered Fake News Detector web application designed to help users evaluate the credibility of news headlines, articles, and URLs in real time. The application integrates IBM watsonx.ai's Granite large language model to analyze submitted text and generate a clear, evidence-based credibility verdict.

The project demonstrates how a generative AI model can be integrated into a web application through an API to solve a real-world problem — the rapid spread of misinformation — by giving users an accessible tool to check news content before trusting or sharing it.

2. Objectives

- Develop a web application that integrates a generative AI model (IBM Granite via watsonx.ai) through an API.
- Analyze news text and URLs to detect signs of misinformation.
- Present AI output in a clear, non-technical way — verdict, confidence score, and explanation.
- Support multiple languages (English and Hindi) with auto-detection.
- Provide a side-by-side comparison tool for evaluating two headlines at once.
- Educate users on how to independently spot fake news.
- Deploy the application for public access using Railway.

3. Technologies Used

- HTML5
- CSS3
- JavaScript (ES6)
- IBM watsonx.ai — Granite large language model (AI analysis engine)
- REST API integration for text/URL analysis and comparison
- Railway (deployment and hosting)

4. Features of the Application

- Analyze News Text: paste a headline, article, or URL (up to 3,000 characters) and get an instant AI credibility verdict.
- Language auto-detection with explicit support for English and Hindi.
- Confidence score showing how certain the AI is in its verdict.
- Highlighted text feature that flags suspicious words and phrases within the submitted content.
- Plain-language explanation and summary of why content was judged credible or not.
- Red flag signals list, calling out specific misinformation indicators found in the text.

- Credibility checklist to help users evaluate the source systematically.
- Compare Two Headlines: submit two headlines or texts side by side for a comparative credibility analysis.
- Built-in "Spotting Fake News" guidance and curated links to fact-checking resources.
- Responsive, user-friendly interface suitable for both desktop and mobile use.

5. Implementation

The front end is built with HTML, CSS, and JavaScript, providing an interface where users can paste text or a URL for analysis. When a request is submitted, the application calls a REST API that forwards the content to IBM watsonx.ai's Granite model.

The model returns a structured response — verdict, confidence score, explanation, highlighted terms, and red flag signals — which the front end parses and renders dynamically. The same pipeline powers the headline comparison feature, which runs both inputs through the model and displays the results side by side. The application is deployed and hosted on Railway for public access.

6. Project Links

Hosted Link (Live Demo)

<https://newsiq-production.up.railway.app/>

GitHub Repository

<https://github.com/manogna4545/NewsIQ.git>

7. Snippets

Frontend — submitting text for analysis (script.js)

```
async function analyzeText(inputText, language = "auto") {
  setLoading(true);
  try {
    const response = await fetch("/api/analyze", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: inputText, language }),
    });

    if (!response.ok) throw new Error(`Analysis failed: ${response.status}`);

    const result = await response.json();
    renderVerdict(result);
  } catch (err) {
    showError("Could not analyze this content. Please try again.");
  } finally {
    setLoading(false);
  }
}
```

Backend — REST endpoint calling watsonx.ai Granite (server.js)

```
app.post("/api/analyze", async (req, res) => {
  const { text, language } = req.body;

  const prompt = buildCredibilityPrompt(text, language);

  const watsonRes = await fetch(
    `${process.env.WATSONX_URL}/ml/v1/text/generation?version=2024-05-01`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${await getAccessToken()}`,
      },
      body: JSON.stringify({
        model_id: "ibm/granite-13b-instruct-v2",
        project_id: process.env.WATSONX_PROJECT_ID,
        input: prompt,
        parameters: { max_new_tokens: 400, temperature: 0.2 },
      }),
    }
  );

  const data = await watsonRes.json();
  const parsed = parseCredibilityResponse(data.results[0].generated_text);

  res.json(parsed);
});
```

Parsing the structured AI response (utils.js)

```
function parseCredibilityResponse(rawText) {
const json = extractJsonBlock(rawText);
  return
{
  verdict: json.verdict,           // "REAL" | "SUSPICIOUS" | "FAKE"
  confidence: json.confidence,     // 0-100
  explanation: json.explanation,
  highlightedTerms: json.flagged_phrases || [],
  redFlags: json.red_flags || [],
};
}
```

8. Commands

Standard commands for setting up, running, and deploying this type of Node.js + watsonx.ai project on Railway:

Local setup

```
# Clone the repository
git clone https://github.com/manogna4545/NewsIQ.git
cd NewsIQ

# Install dependencies
npm install

# Configure environment variables
cp .env.example .env
# then set WATSONX_API_KEY, WATSONX_PROJECT_ID, WATSONX_URL

# Run the app locally npm start
```

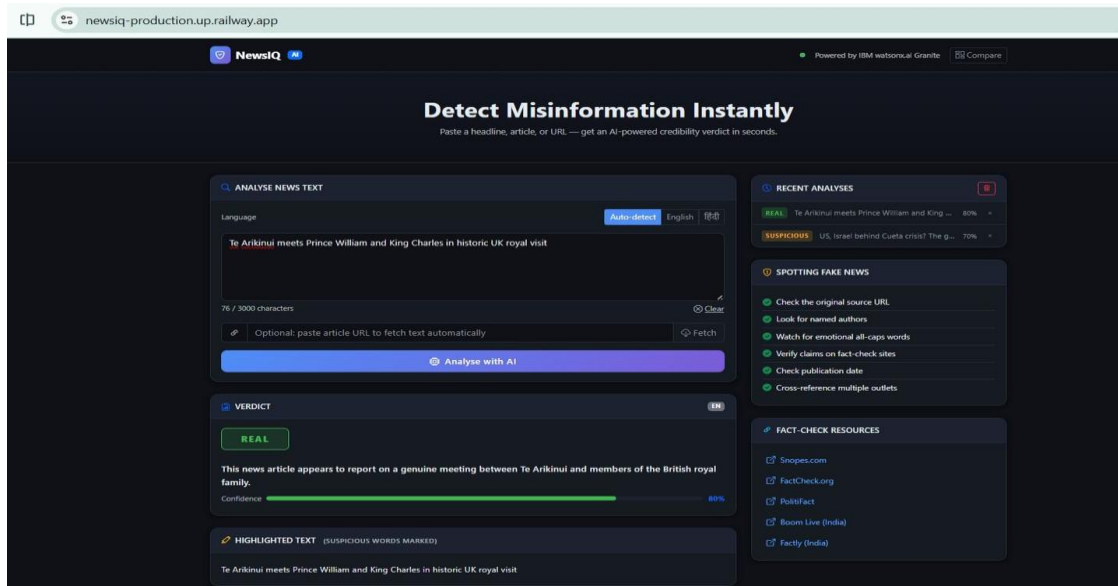
Deploy to Railway

```
# Install the Railway CLI (one-time)
npm install -g @railway/cli

# From the project root
railway login
railway init
railway up

# Set environment variables on Railway
railway variables set WATSONX_API_KEY=your_key_here
railway variables set WATSONX_PROJECT_ID=your_project_id
```

9. Screenshots



10. AI Model / API Used

Model: IBM watsonx.ai — Granite

Purpose: To analyze news text or URLs and generate a credibility verdict, confidence score, explanation, and misinformation indicators.

Capabilities Used:

- Credibility verdict generation with confidence scoring.
- Suspicious phrase / red-flag detection within submitted text.
- Plain-language explanation generation.
- Comparative analysis of two separate headlines or texts.

11. Learning Outcomes

Through this project, practical experience was gained in:

- Integrating a generative AI model (IBM Granite via watsonx.ai) into a web application through an API.
- Designing prompts and handling structured AI responses (verdict, confidence, explanation, flags).
- Handling asynchronous API calls, loading states, and error handling in JavaScript.
- Building a multi-language (English/Hindi) user interface with auto-detection.
- Designing a responsive, user-friendly front end for an AI-driven tool.
- Deploying a full web application using Railway.

12. Conclusion

NewsIQ demonstrates the practical application of generative AI in tackling a real-world problem — the spread of misinformation. By integrating IBM watsonx.ai's Granite model through a REST API, the application delivers instant, explainable credibility assessments for news content in multiple languages.

This project strengthened understanding of AI/API integration, asynchronous programming, structured response handling, and deployment of AI-powered web applications for public use.